

Perceptron, Adaline & Logistic Regression

COMP3314
Machine Learning

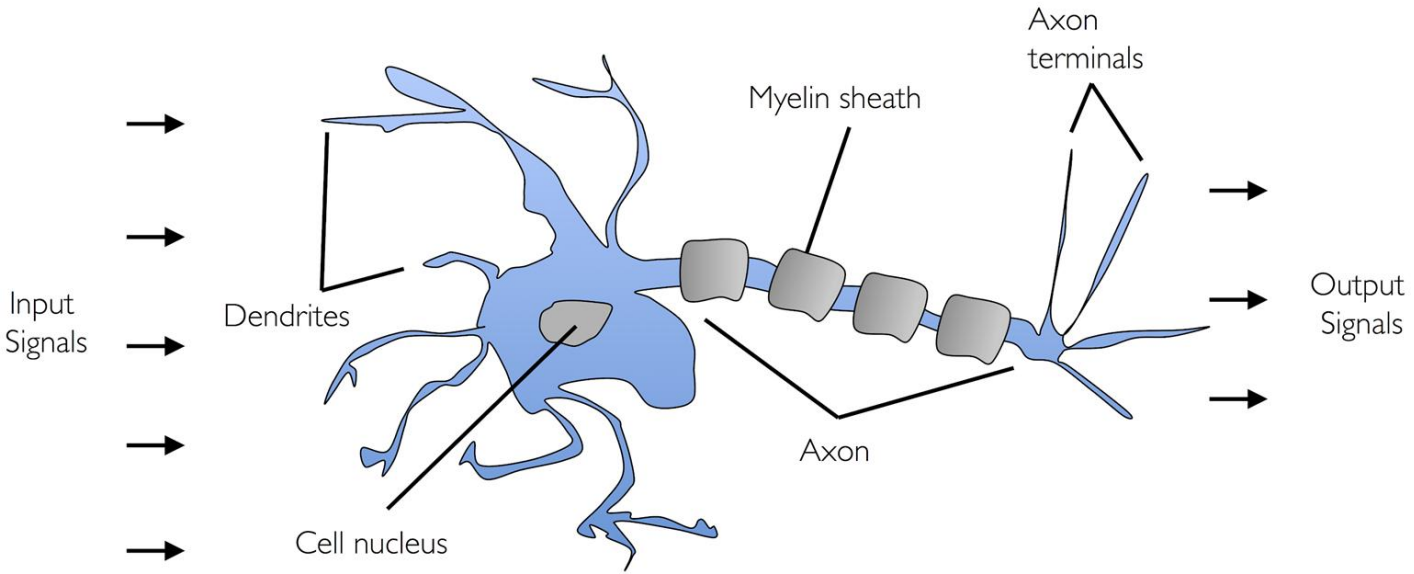
Dong Xu

Outline

- Neuron & Artificial Neuron
- Perceptron
  - Perceptron Learning Rule
  - Linearly Separable
  - Implementation
- Adaptive Linear Neuron (Adaline)
  - Implementation
  - Feature Scaling
  - Stochastic Gradient Descent / Mini-Batch Learning
- Logistic Regression

Neuron

- Neurons are interconnected nerve cells in the brain that are involved in the processing and transmitting of chemical and electrical signals



Artificial Neuron - History

- McCulloch and Pitts described the first artificial neuron in 1943 (aka MCP neuron) as a simple logic gate with binary outputs
  - Signals arrive at the dendrites, are integrated into the cell body, and, if the accumulated signal exceeds a certain threshold, an output signal is generated at the axon
- A few years later, in 1958, Rosenblatt published the first concept of the perceptron learning rule based on the MCP neuron
  - Automatically learn the optimal weight coefficients that are then multiplied with the input features

Artificial Neuron - Definition

- Consider a binary classification task where we refer to our two classes as 1 (positive class) and -1 (negative class)
- We can define a decision function

$$\phi(z) = \begin{cases} 1 & if\ z \geq \theta \\ -1 & otherwise \end{cases}$$

where θ is a threshold and z is the net input

$$z = w_1 x_1 + \ldots + w_m x_m$$

of the input values **x** and the corresponding weight vector **w**

$$\boldsymbol{w} = \begin{bmatrix} w_1 \\ \vdots \\ w_m \end{bmatrix}, \quad \boldsymbol{x} = \begin{bmatrix} x_1 \\ \vdots \\ x_m \end{bmatrix}$$

Artificial Neuron - Definition

- For simplicity, we can bring the threshold θ to the left side of the equation and define

$$z = w_0 x_0 + w_1 x_1 + \ldots + w_m x_m = \boldsymbol{w}^T \boldsymbol{x}$$

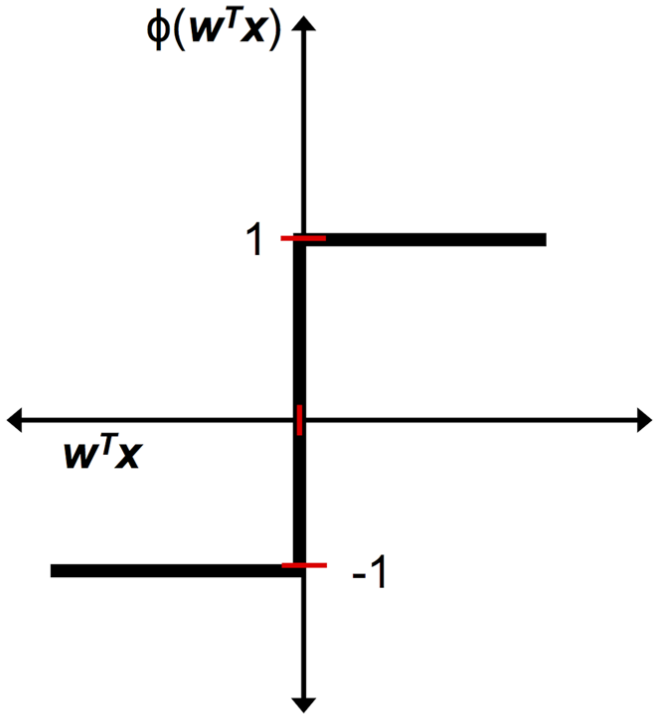
and

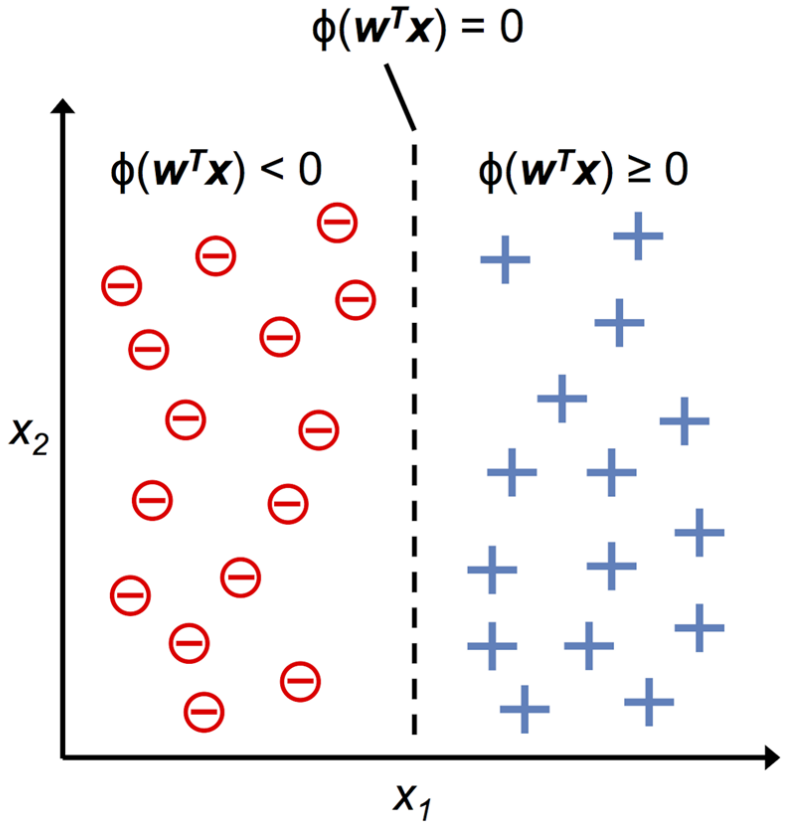
$$\phi(z) = \begin{cases} 1 & \text{if } z \geq 0 \\ -1 & \text{otherwise} \end{cases}$$

where $w_0 = -\theta$ and $x_0 = 1$

- w_0 is called the bias unit

Artificial Neuron - Definition





Outline

- Neuron & Artificial Neuron
- Perceptron
  - Perceptron Learning Rule
  - Linearly Separable
  - Implementation
- Adaptive Linear Neuron (Adaline)
  - Implementation
  - Feature Scaling
  - Stochastic Gradient Descent / Mini-Batch Learning
- Logistic Regression

Perceptron Learning Rule

- Rosenblatt's perceptron rule can be summarized by the following steps
  a. Initialize the weights
  b. For each training sample $x^{(i)}$
     i. Compute the output value $\hat{y}$,
     i.e. the class label predicted by unit step function
     i. Update the weights

$$w_j := w_j + \Delta w_j$$

The value of Δw_j is calculated as follows

$$\Delta w_j = \eta \left(y^{(i)} - \hat{y}^{(i)} \right) x_j^{(i)}$$

Where η is the learning rate, $y^{(i)}$ is the true class label of the ith training sample, and $\hat{y}^{(i)}$ is the predicted class label

Weight Update

- All weights in the weight vector are updated simultaneously, which means that we don't recompute the $\hat{y}^{(i)}$ before all of the weights Δw_j are updated
  - I.e., for a two-dimensional dataset, we would write the update as

$$\Delta w_0 = \eta \left(y^{(i)} - output^{(i)} \right)$$

$$\Delta w_1 = \eta \left(y^{(i)} - output^{(i)} \right) x_1^{(i)}$$

$$\Delta w_2 = \eta \left(y^{(i)} - output^{(i)} \right) x_2^{(i)}$$

Weight Update

- In the two scenarios where the perceptron predicts the class label correctly, the weights remain unchanged

$$\Delta w_j = \eta\left(-1-\left(-1\right)\right)x_j^{(i)} = 0$$

$$\Delta w_j = \eta\left(1-1\right)x_j^{(i)} = 0$$

- However, in the case of a wrong prediction, the weights are being pushed towards the direction of the positive or negative target class

$$\Delta w_j = \eta\left(1--1\right)x_j^{(i)} = \eta\left(2\right)x_j^{(i)}$$

$$\Delta w_j = \eta\left(-1-1\right)x_j^{(i)} = \eta\left(-2\right)x_j^{(i)}$$

Weight Update

- Note that the weight update is proportional the value of $x_j^{(i)}$

$$\Delta w_j = \eta \left(y^{(i)} - \hat{y}^{(i)} \right) x_j^{(i)}$$

Perceptron - Implementation

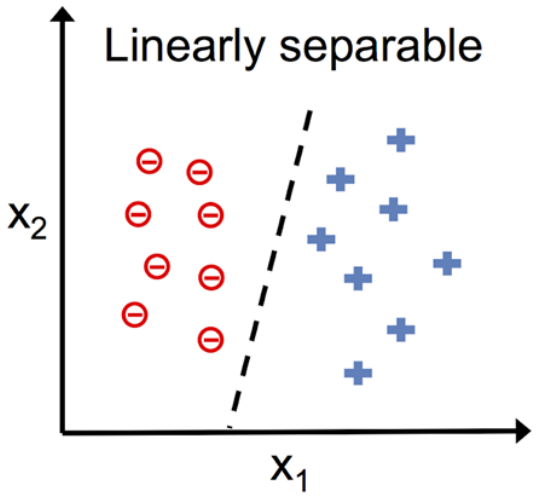
$$\Delta w_j = \eta \left(y^{(i)} - \hat{y}^{(i)} \right) x_j^{(i)}$$

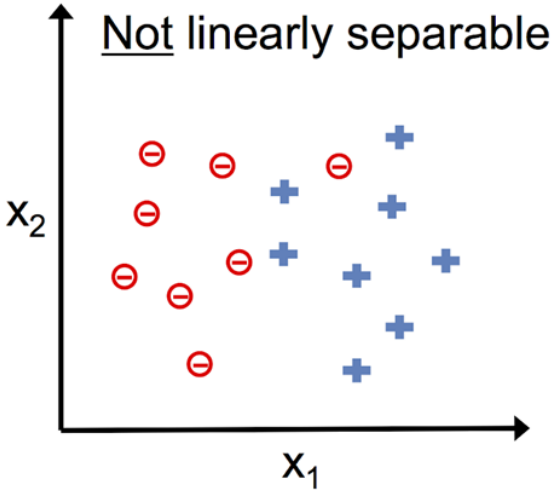
```python
import numpy as np
class Perceptron(object):
    def __init__(self, eta=0.01, n_iter=50, random_state=1):
        self.eta = eta
        self.n_iter = n_iter
        self.random_state = random_state
    def fit(self, X, y):
        rgen = np.random.RandomState(self.random_state)
        self.w_ = rgen.normal(loc=0.0, scale=0.01, size=1 + X.shape[1])
        self.errors_ = []
        for _ in range(self.n_iter):
            errors = 0
            for xi, target in zip(X, y):
                update = self.eta * (target - self.predict(xi))
                self.w_[1:] += update * xi
                self.w_[0] += update
                errors += int(update != 0.0)
            self.errors_.append(errors)
        return self
    def net_input(self, X):
        return np.dot(X, self.w_[1:]) + self.w_[0]
    def predict(self, X):
        return np.where(self.net_input(X) >= 0.0, 1, -1)
ppn = Perceptron(eta=0.1, n_iter=10)
ppn.fit(X, y)
```

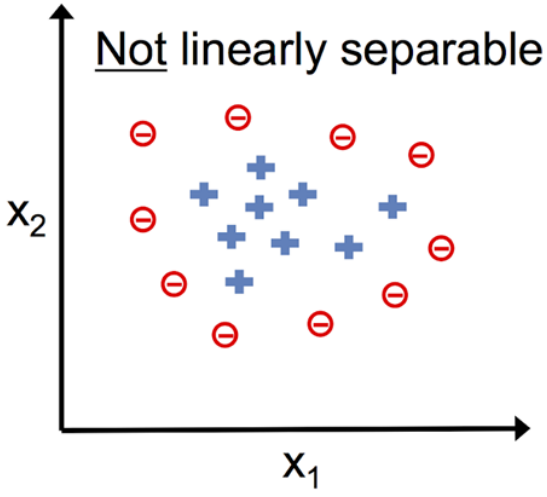
Linearly Separable

- It was shown that the convergence of the perceptron is only guaranteed if the two classes are linearly separable and the learning rate is sufficiently small
- If the two classes can't be separated by a linear decision boundary, we can set a maximum number of passes over the training dataset (epochs) and/or a threshold for the number of tolerated misclassifications
  - The perceptron would never stop updating the weights otherwise

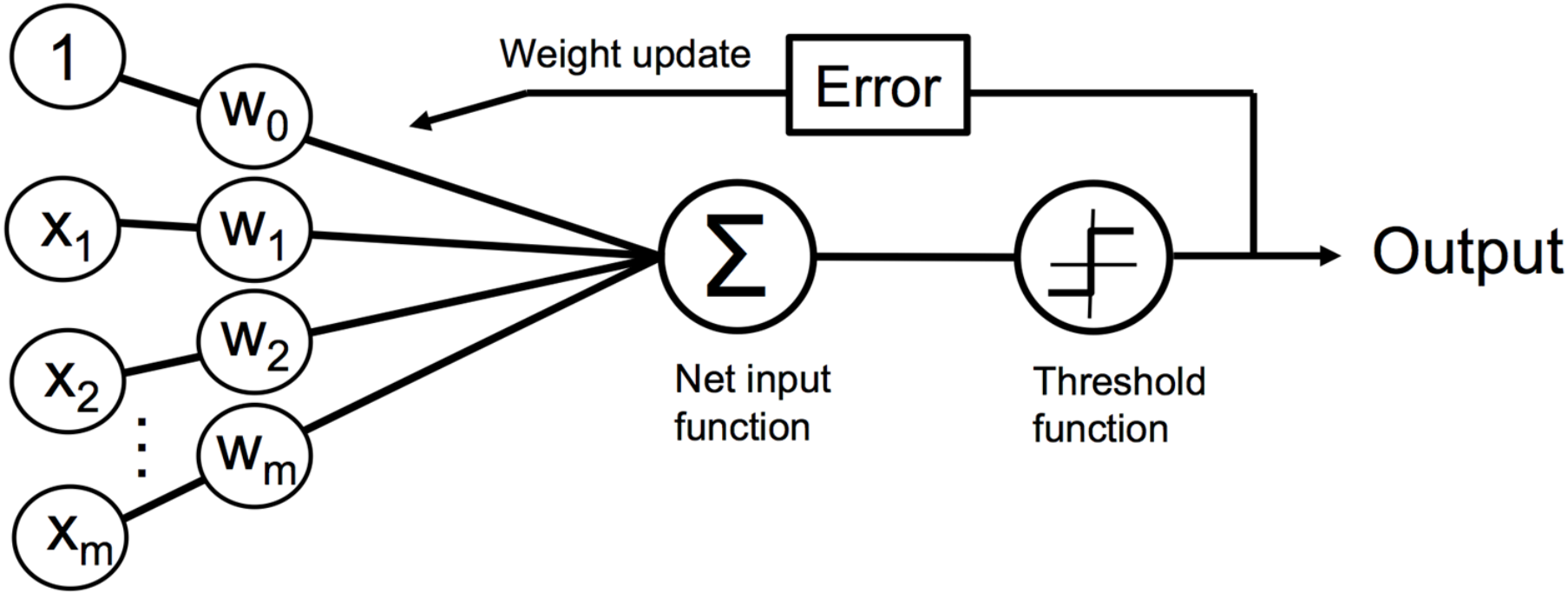
Linearly Separable vs. Not Linearly Separable







Perceptron


1
w_0
x_1
w_1
w_2
x_2
w_m
x_m
Weight update
Error
Σ
Net input function
Threshold function
Output

Outline

- Neuron & Artificial Neuron
- Perceptron
  - Perceptron Learning Rule
  - Linearly Separable
  - Implementation
- Adaptive Linear Neuron (Adaline)
  - Implementation
  - Feature Scaling
  - Stochastic Gradient Descent / Mini-Batch Learning
- Logistic Regression

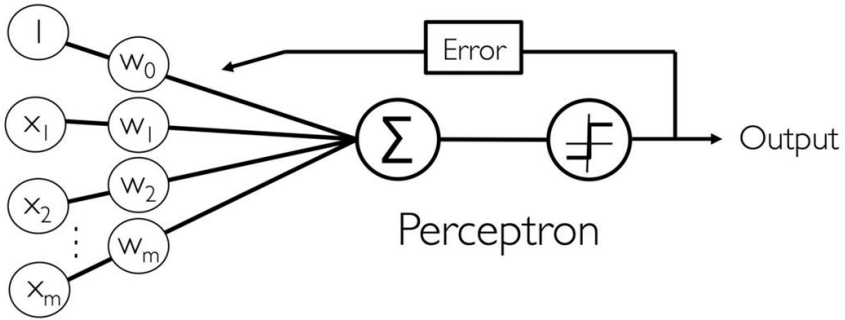
Adaptive Linear Neuron

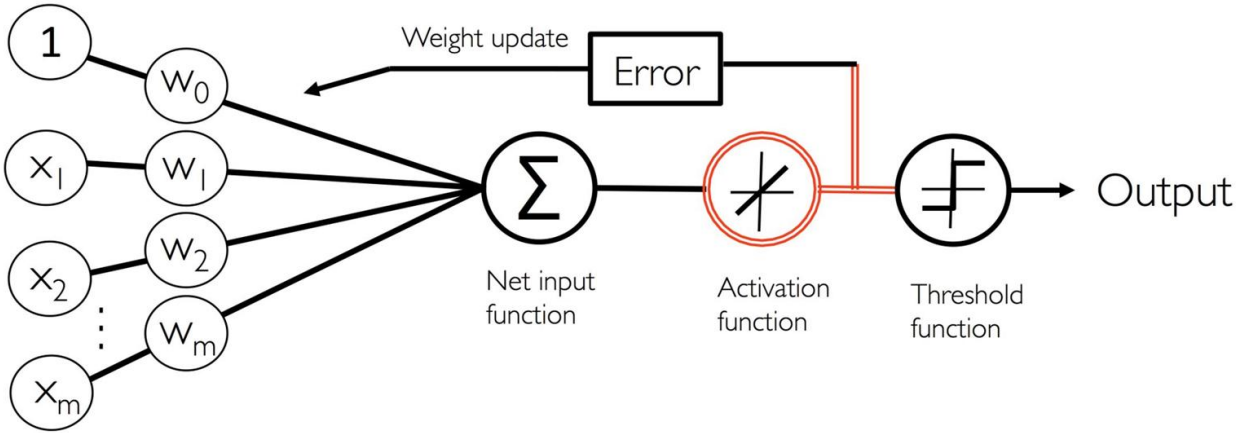
- ADAptive LInear NEuron: Adaline
- Improvement on Perceptron algorithm
- Published in 1960 by Bernard Widrow and Ted Hoff
- In Adaline the weights are updated based on a linear activation function

$$\phi\left(\boldsymbol{w}^T \boldsymbol{x}\right) = \boldsymbol{w}^T \boldsymbol{x}$$

- While the linear activation function is used for learning the weights, we still use a threshold function to make the final prediction

Adaptive Linear Neuron





Adaptive Linear Neuron (Adaline)

Objective Function

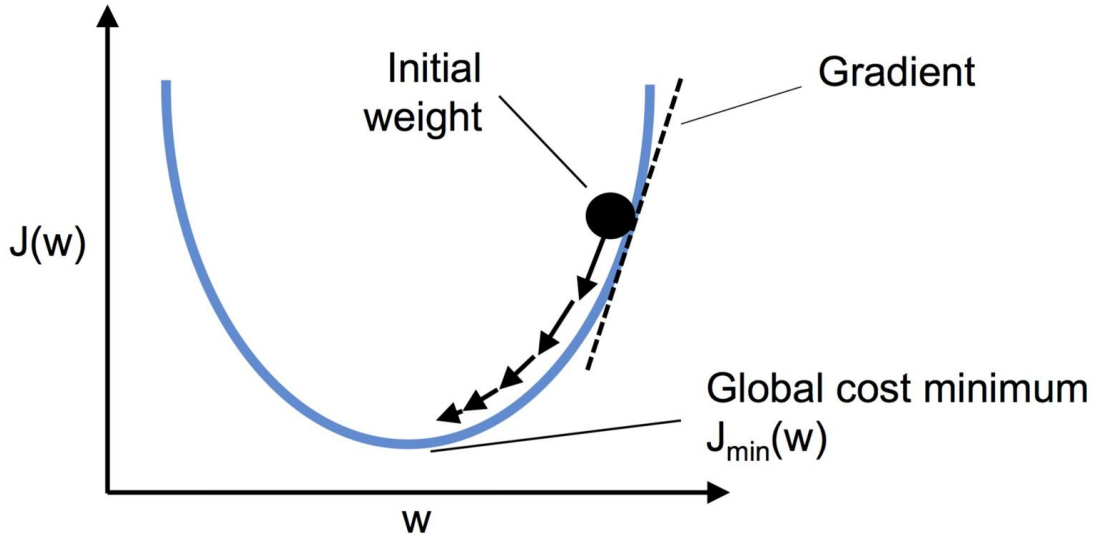
- One of the key ingredients of supervised ML algorithms is the objective function that is to be optimized
  - E.g., cost function that we want to minimize
- In Adaline the cost function J is defined as the sum of squared errors (SSE) between the calculated and true class label

$$J(\boldsymbol{w}) = \frac{1}{2}\sum_i \left(y^{(i)} - \phi\left(z^{(i)}\right)\right)^2$$

- It can be shown that J is differentiable and convex
  - It will be easy to minimize (using e.g. gradient descent)

Gradient Descent

- A generic optimization algorithm
  - Capable of finding optimal solutions to a wide range of problems
- Main idea
  - Tweak parameters iteratively in order to minimize a cost function
  - Measures the local gradient of the error function with regard to the parameter vector and goes in the direction of descending gradient
    - Once the gradient is zero, it has reached a minimum



Weight Update

- Update the weights by taking a step in the opposite direction of the gradient $\nabla J(\boldsymbol{w})$ of our cost function $J(\boldsymbol{w})$

$$\boldsymbol{w} := \boldsymbol{w} + \Delta \boldsymbol{w}$$

- The weight change $\mathbf{\Delta w}$ is defined as the negative gradient multiplied by the learning rate η

$$\Delta \boldsymbol{w} = -\eta \nabla \mathrm{J}\left(\boldsymbol{w}\right)$$

$$\frac{\partial J}{\partial w_j} = \frac{\partial}{\partial w_j} \frac{1}{2} \sum_i \left(y^{(i)} - \phi\left(z^{(i)} \right) \right)^2$$

$$= \frac{1}{2} \frac{\partial}{\partial w_j} \sum_i \left(y^{(i)} - \phi\left(z^{(i)} \right) \right)^2$$

$$= \frac{1}{2} \sum_i 2 \left(y^{(i)} - \phi\left(z^{(i)} \right) \right) \frac{\partial}{\partial w_j} \left(y^{(i)} - \phi\left(z^{(i)} \right) \right)$$

$$= \sum_i \left(y^{(i)} - \phi\left(z^{(i)} \right) \right) \frac{\partial}{\partial w_j} \left(y^{(i)} - \sum_k \left(w_k x_k^{(i)} \right) \right)$$

$$= \sum_i \left(y^{(i)} - \phi\left(z^{(i)} \right) \right) \left(-x_j^{(i)} \right)$$

$$= -\sum_i \left(y^{(i)} - \phi\left(z^{(i)} \right) \right) x_j^{(i)}$$

Weight Update

- To compute the gradient of the cost function, we compute the partial derivative of the cost function with respect to each weight w_j

$$\frac{\partial J}{\partial w_j} = -\sum_i \left(y^{(i)} - \phi\left(z^{(i)} \right) \right) x_j^{(i)}$$

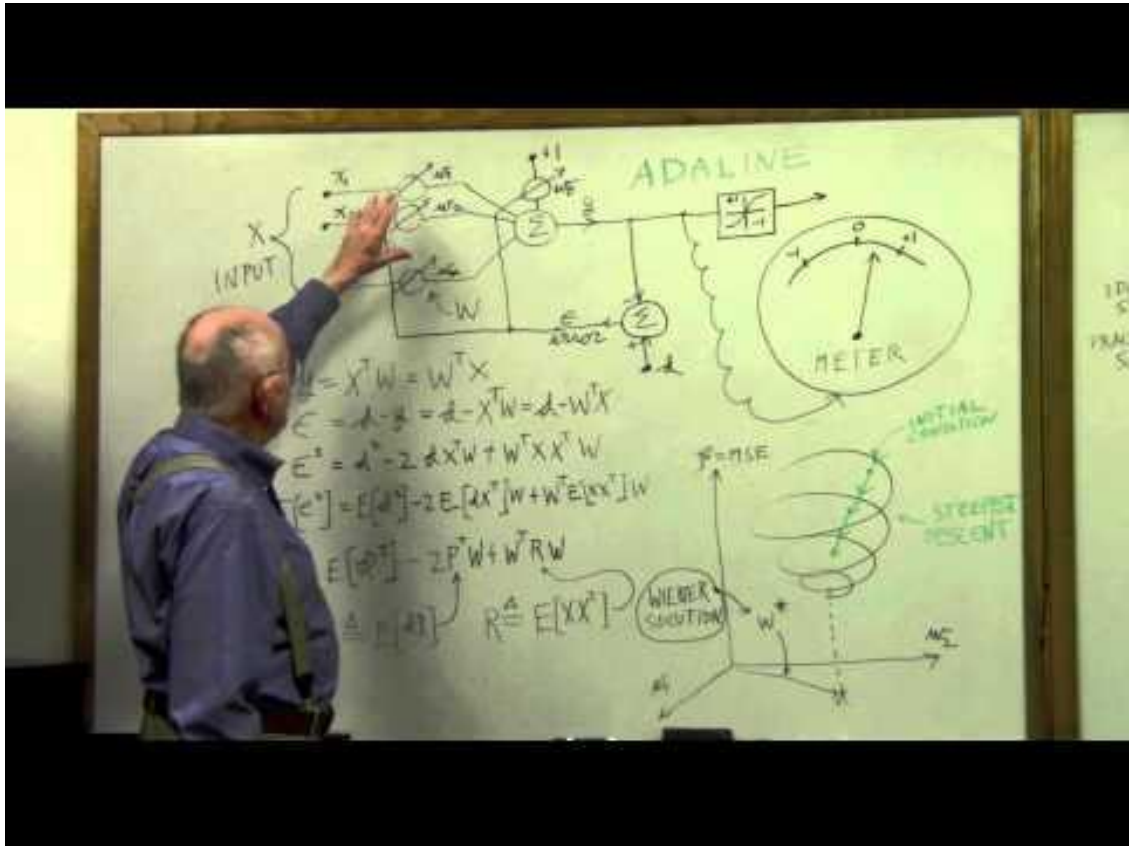
- The update of weight w_j can then be written as

$$\Delta w_j = -\eta \frac{\partial J}{\partial w_j} = \eta \sum_i \left(y^{(i)} - \phi\left(z^{(i)} \right) \right) x_j^{(i)}$$

- Batch gradient descent
  - Note that the weight update is calculated based on all samples in the training set (instead of updating the weights incrementally after each sample)

Learn from the Author

- Bernard Widrow and Ted Hoff published a paper introducing Adaline in 1959
- Bernard Widrow has a youtube channel
  - He has a video talking about Adaline :-)



Adaline - Implementation

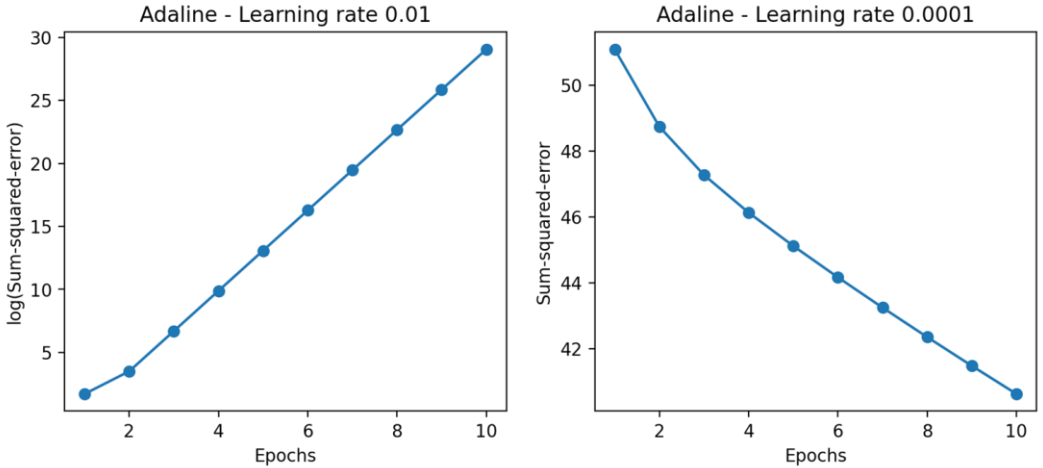
```
class AdalineGD(object):
    def __init__(self, eta=0.01, n_iter=50, random_state=1):
        self.eta = eta
        self.n_iter = n_iter
        self.random_state = random_state
    def fit(self, X, y):
        rgen = np.random.RandomState(self.random_state)
        self.w_ = rgen.normal(loc=0.0, scale=0.01, size=1 + X.shape[1])
        self.cost_ = []
        for i in range(self.n_iter):
            net_input = self.net_input(X)
            output = self.activation(net_input)
            errors = (y - output)
            self.w_[1:] += self.eta * X.T.dot(errors)
            self.w_[0] += self.eta * errors.sum()
            cost = (errors**2).sum() / 2.0
            self.cost_.append(cost)
        return self
    def net_input(self, X):
        return np.dot(X, self.w_[1:]) + self.w_[0]
    def activation(self, X):
        return X
    def predict(self, X):
        return np.where(self.activation(self.net_input(X)) >= 0.0, 1, -1)
ada = AdalineGD(n_iter=30, eta=0.0001).fit(X, y)
```

$$\Delta w_j = -\eta \frac{\partial J}{\partial w_j} = \eta \sum_i \left(y^{(i)} - \phi\left(z^{(i)}\right) \right) x_j^{(i)}$$

$$J(\boldsymbol{w}) = \frac{1}{2} \sum_i \left(y^{(i)} - \phi\left(z^{(i)}\right) \right)^2$$

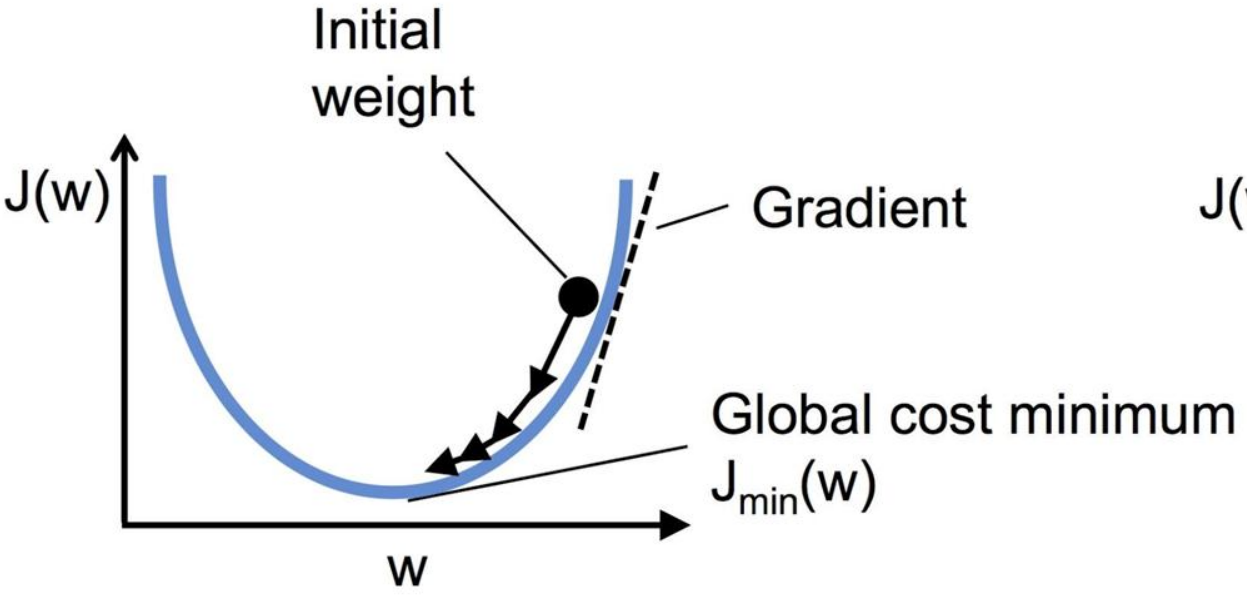
Code is here

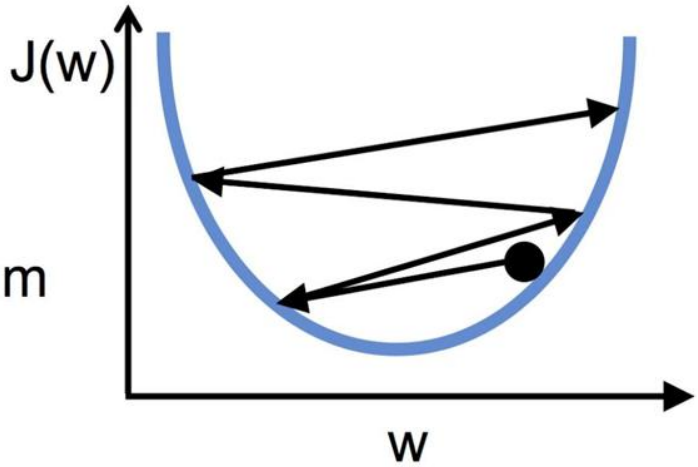
```
import matplotlib.pyplot as plt
fig, ax = plt.subplots(nrows=1, ncols=2, figsize=(10, 4))
ada1 = AdalineGD(n_iter=10, eta=0.01).fit(X, y)
ax[0].plot(range(1, len(ada1.cost_) + 1), np.log10(ada1.cost_), marker='o')
ax[0].set_xlabel('Epochs')
ax[0].set_ylabel('log(Sum-squared-error)')
ax[0].set_title('Adaline - Learning rate 0.01')
ada2 = AdalineGD(n_iter=10, eta=0.0001).fit(X, y)
ax[1].plot(range(1, len(ada2.cost_) + 1), ada2.cost_, marker='o')
ax[1].set_xlabel('Epochs')
ax[1].set_ylabel('Sum-squared-error')
ax[1].set_title('Adaline - Learning rate 0.0001')
plt.show()
```



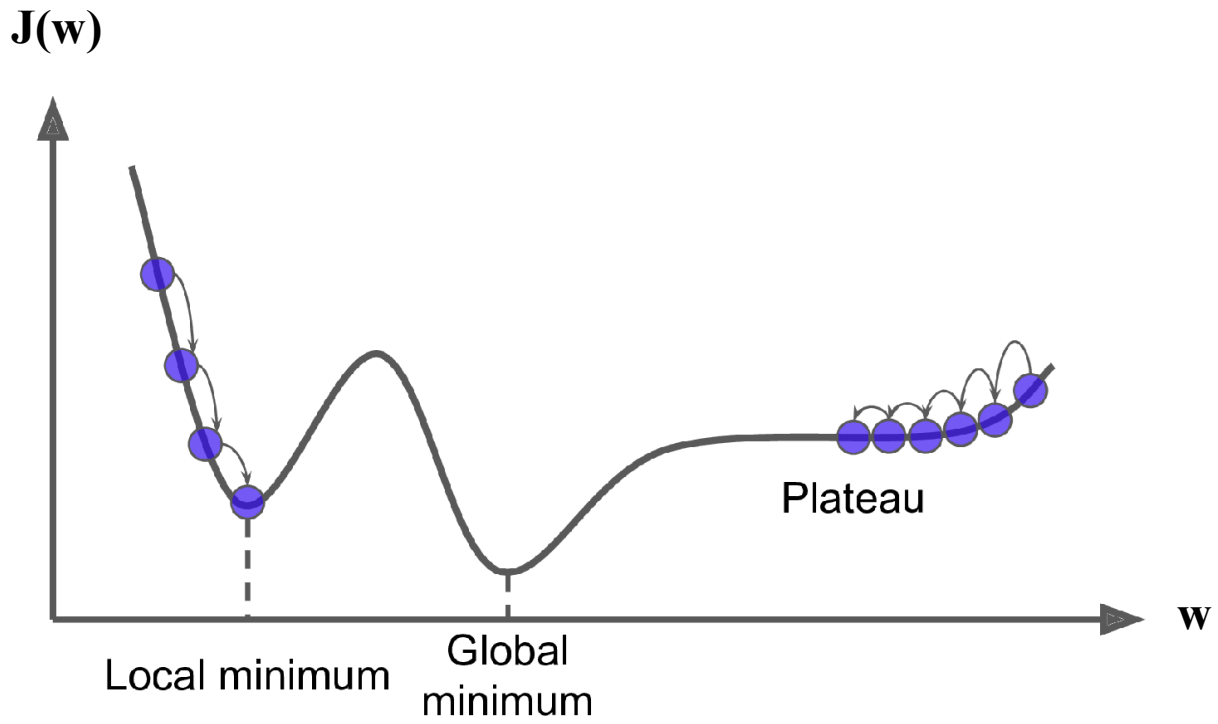
Learning Rate

- Important hyperparameter of Gradient Descent
- If too small
  - Requires excessive many iterations to converge
    - Takes a long time
- If too large
  - Might jump across the valley and end up on the other side
    - Possibly even higher up than before
    - Algorithm may diverge, with larger and larger values


Initial
weight
J(w)
Gradient
Global cost minimum
$J_{min}(w)$
w

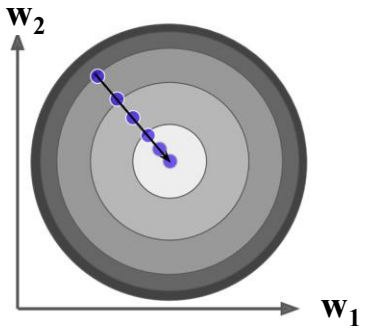

J(w)
w

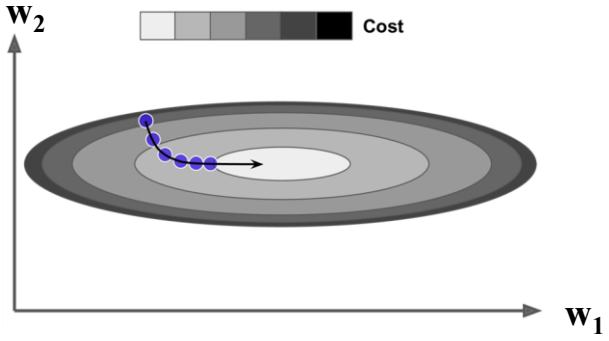
Gradient Descent Pitfalls



Feature Scaling

- Many ML algorithms require feature scaling for optimal performance
  - E.g., gradient descent converges more quickly if our data follows a standard distribution
- Standardization
  - A feature scaling method
  - After standardization, feature have
    - a mean value of 0
    - a standard deviation of 1





Standardization

- For instance, to standardize the *j*th feature, we can simply subtract the sample mean μ_j from every training sample and divide it by its standard deviation σ_j

$$\boldsymbol{x}'_j = \frac{\boldsymbol{x}_j - \mu_j}{\sigma_j}$$

- Here, $\boldsymbol{x}_j$ is a vector consisting of the *j*th feature values of all training samples n
- This standardization technique is applied to each feature *j* in our dataset

Stochastic Gradient Descent

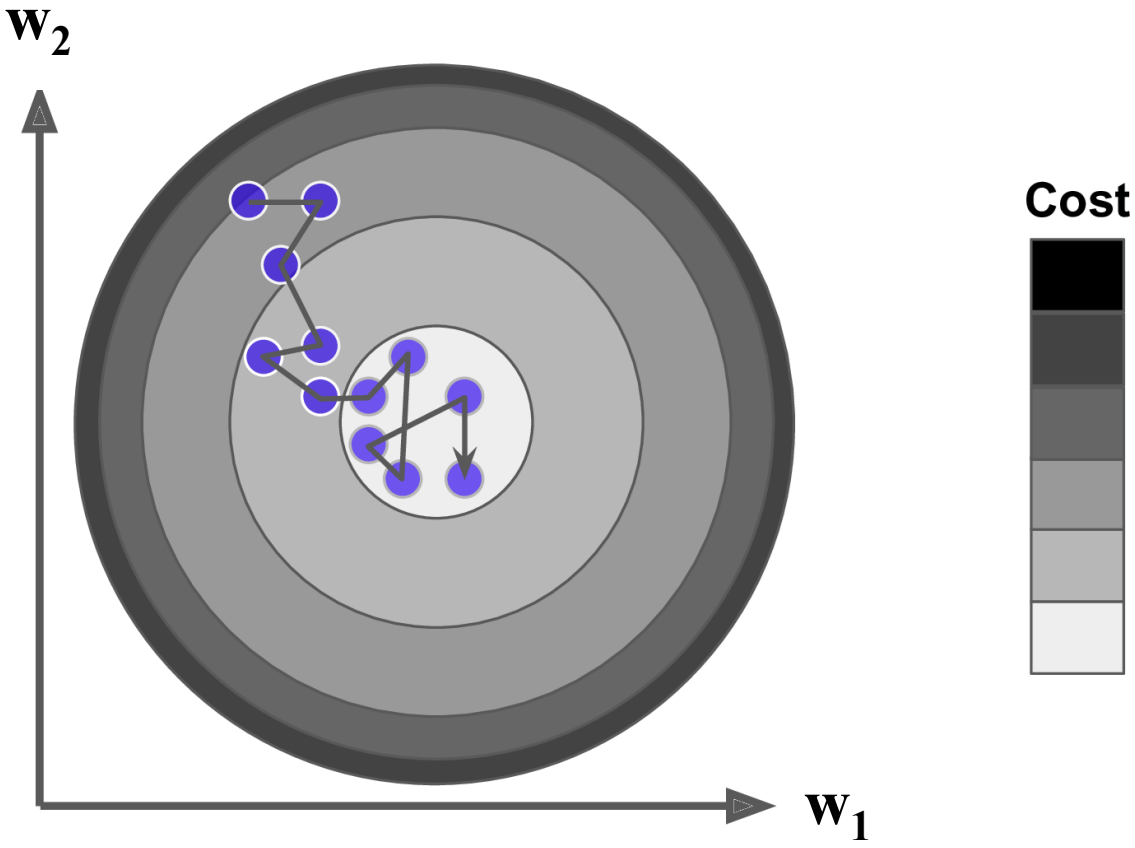
- Imagine we have a very large dataset with millions of data points
  - Not uncommon in ML applications
  - Running batch gradient descent can be computationally costly in such scenarios since we need to reevaluate the whole training dataset each time we take one step
- Stochastic Gradient Descent is a popular alternative
  - Instead of updating the weights based on the sum of the accumulated errors over all samples $\boldsymbol{x}^{(i)}$

$$\Delta \boldsymbol{w} = \eta \sum_i \left(y^{(i)} - \phi\left(z^{(i)} \right) \right) \boldsymbol{x}^{(i)}$$

  we update all the weights incremental for each training sample

Stochastic Gradient Descent

- Typically reaches convergences faster because of the more frequent weight updates
- Can escape shallow local minima more readily if we are working with nonlinear cost functions
- It is important to present it training data in a random order
- In addition shuffle the training set for every epoch to prevent cycles
- Another advantage of stochastic gradient descent is that we can use it for online learning
    - In online learning, our model is trained on the fly as new training data arrives


w_2
w_1
Cost

Mini-Batch Learning

- A compromise between batch gradient descent and stochastic gradient descent
- Apply batch gradient descent to smaller subsets of the training data
  - E.g., 32 samples at a time
- The advantage over batch gradient descent is that convergence is reached faster via mini-batches because of the more frequent weight updates
- Furthermore, mini-batch learning allows us to replace the for loop over the training samples in stochastic gradient descent with vectorized operations

Outline

- Neuron & Artificial Neuron
- Perceptron
  - Perceptron Learning Rule
  - Linearly Separable
  - Implementation
- Adaptive Linear Neuron (Adaline)
  - Implementation
  - Feature Scaling
  - Stochastic Gradient Descent / Mini-Batch Learning
- Logistic Regression

Logistic Regression - Intuition

- Widely used algorithm for binary classification
  - For classification, not regression
- The logit function is defined as the logarithm of odds, i.e.,

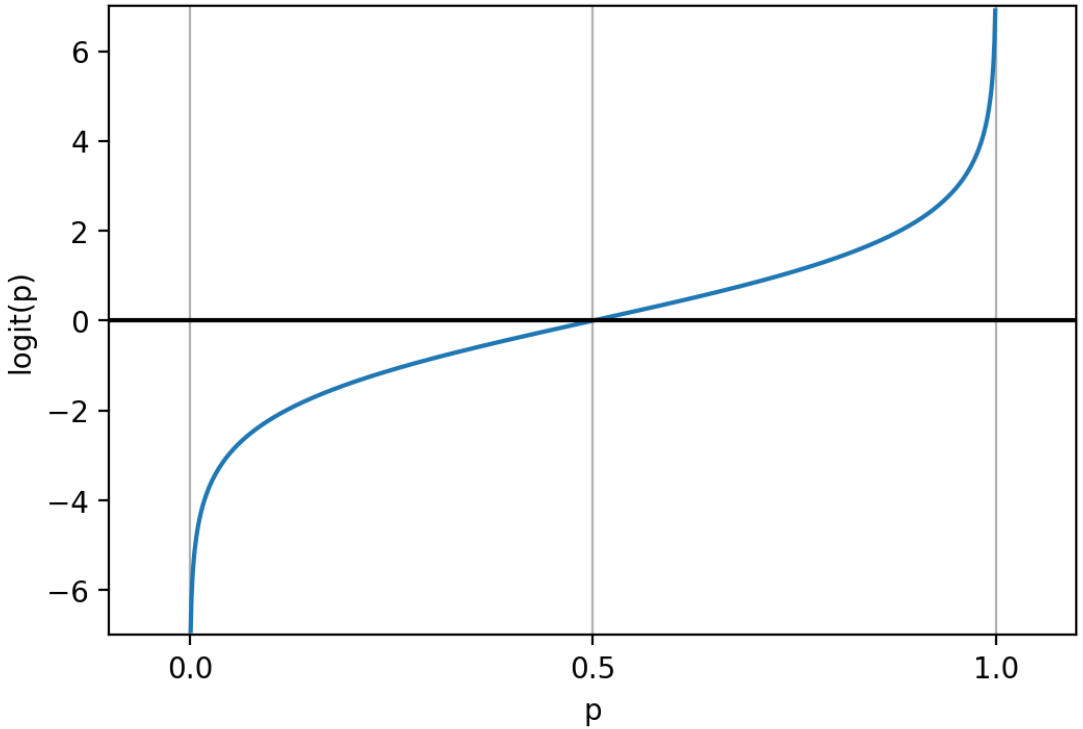
$$logit(p) = log \frac{p}{(1-p)}$$

- It takes as input values in the range 0 to 1 and transforms them to values over the entire real-number range

Logit Function

```
import matplotlib.pyplot as plt
import numpy as np
def logit(p):
    return np.log(p / (1-p))
p = np.arange(0.001, 1, 0.001)
lp = logit(p)
plt.plot(p, lp)
plt.axhline(0, color='k')
plt.xlim(-0.1, 1.1)
plt.ylim(-7, 7)
plt.xlabel('p')
plt.ylabel('logit(p)')
plt.xticks([0.0, 0.5, 1.0])
ax = plt.gca()
ax.xaxis.grid(True)
plt.show()
```

$$logit(p) = log \frac{p}{(1-p)}$$

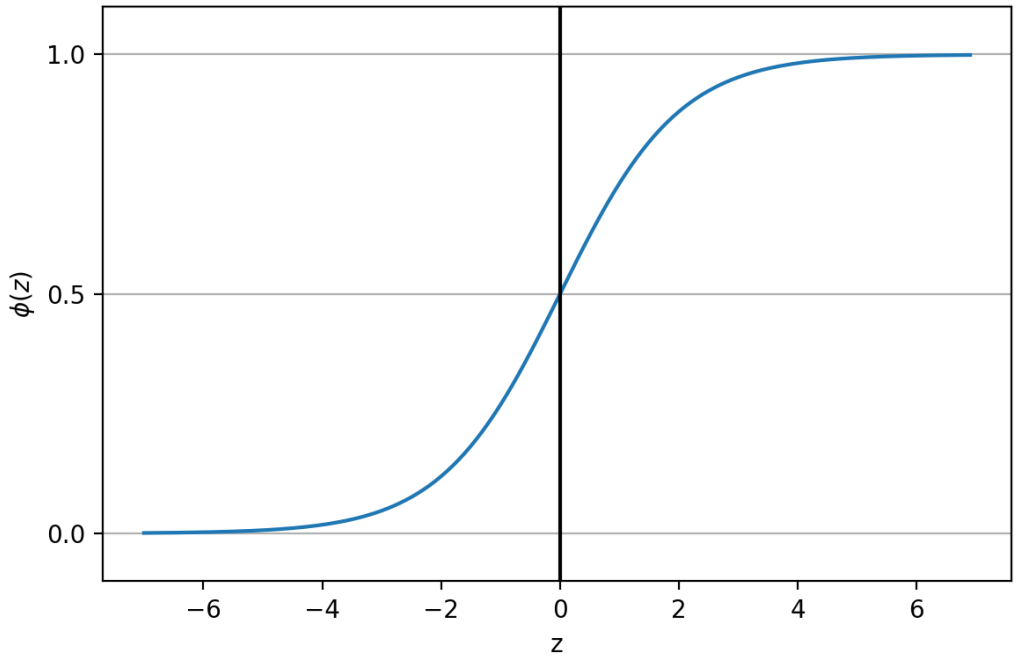


Inverse of Logit (Sigmoid) Function

$$logit(p) = log\frac{p}{(1-p)}$$

$$logit^{-1}(z) = \phi(z) = \frac{1}{1+e^{-z}} = \frac{e^{z}}{1+e^{z}}$$

```
def sigmoid(z):
    return 1.0 / (1.0 + np.exp(-z))
z = np.arange(-7, 7, 0.1)
phi_z = sigmoid(z)
plt.plot(z, phi_z)
plt.axvline(0.0, color='k')
plt.ylim(-0.1, 1.1)
plt.xlabel('z')
plt.ylabel('$\phi (z)$')
plt.yticks([0.0, 0.5, 1.0])
ax = plt.gca()
ax.yaxis.grid(True)
plt.tight_layout()
plt.show()
```



Logistic Regression - Intuition

- The output of the logit function can be used to express a linear relationship between feature values and the log-odds

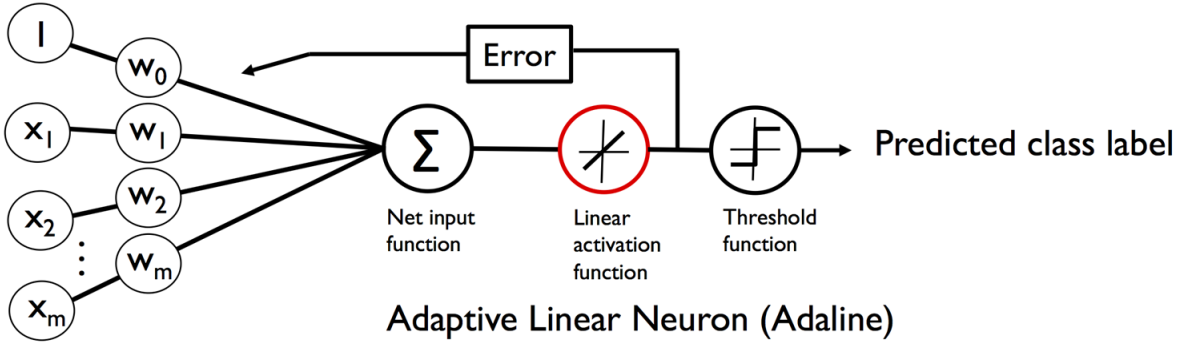
$$logit\left(p\left(y=1\,|\,\boldsymbol{x}\right)\right)=w_0x_0+w_1x_1+\cdots+w_mx_m=\sum_{i=0}^{m}w_ix_i=\boldsymbol{w}^T\boldsymbol{x}$$

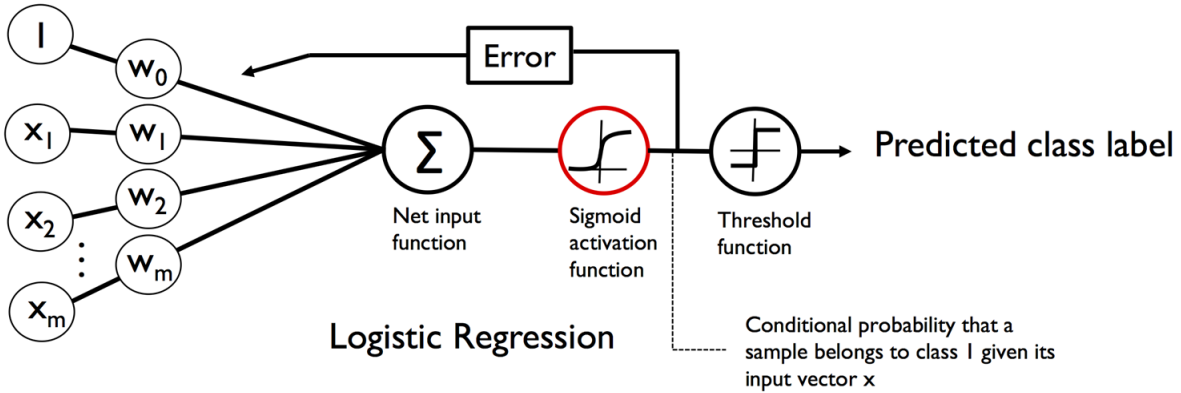
- p (y = 1| **x**) is the conditional probability that a particular sample belongs to class 1 given its features **x**

$$\phi\left(z\right)=\frac{1}{1+e^{-z}}$$

$$z=\boldsymbol{w}^T\boldsymbol{x}=w_0x_0+w_1x_1+\cdots+w_mx_m$$

Adaline vs. Logistic Regression





Output of Sigmoid Function

- The output of the sigmoid function is interpreted as the probability of a particular sample belonging to class 1
  $\phi(z) = P(y = 1 | \mathbf{x}; \mathbf{w})$, given its features $\mathbf{x}$ parameterized by the weights $\mathbf{w}$
  - E.g, let $\phi(z) = 0.8$ for a particular flower sample. I.e., the probability that this sample is an Iris-versicolor flower is 80 %
  - The probability that this flower is an Iris-setosa flower can be calculated as $P(y = 0 | \mathbf{x}; \mathbf{w}) = 1 - P(y = 1 | \mathbf{x}; \mathbf{w}) = 0.2$
    - The predicted probability can then simply be converted into a binary outcome via a threshold function

$$\hat{y} = \begin{cases} 1 & if\ \phi(z) \geq 0.5 \\ 0 & otherwise \end{cases}$$

Learning **w**

- Recall that we defined the sum-squared-error cost function as follows

$$J(\boldsymbol{w}) = \sum_i \frac{1}{2}\left(\phi\left(z^{(i)}\right) - y^{(i)}\right)^2$$

  - We minimized this function in order to learn the weights **w** for our Adaline classification model
- For logistic regression, we define the likelihood L that we want to maximize as

$$L(\boldsymbol{w}) = P(\boldsymbol{y} \mid \boldsymbol{x}; \boldsymbol{w}) = \prod_{i=1}^{n} P\left(y^{(i)} \mid \boldsymbol{x}^{(i)}; \boldsymbol{w}\right) = \prod_{i=1}^{n} \left(\phi\left(z^{(i)}\right)\right)^{y^{(i)}} \left(1 - \phi\left(z^{(i)}\right)\right)^{1-y^{(i)}}$$

Learning **w**

$$L(\boldsymbol{w}) = P(\boldsymbol{y} \mid \boldsymbol{x}; \boldsymbol{w}) = \prod_{i=1}^{n} P\left(y^{(i)} \mid \boldsymbol{x}^{(i)}; \boldsymbol{w}\right) = \prod_{i=1}^{n} \left(\phi\left(z^{(i)}\right)\right)^{y^{(i)}} \left(1 - \phi\left(z^{(i)}\right)\right)^{1-y^{(i)}}$$

- In practice, it is easier to maximize the (natural) log of this equation, which is called the log-likelihood function

$$l(\boldsymbol{w}) = \log L(\boldsymbol{w}) = \sum_{i=1}^{n} \left[y^{(i)} \log\left(\phi\left(z^{(i)}\right)\right) + \left(1 - y^{(i)}\right) \log\left(1 - \phi\left(z^{(i)}\right)\right)\right]$$

- Let's rewrite the log-likelihood as a cost function that can be minimized using gradient descent

$$J(\boldsymbol{w}) = \sum_{i=1}^{n} \left[-y^{(i)} \log\left(\phi\left(z^{(i)}\right)\right) - \left(1 - y^{(i)}\right) \log\left(1 - \phi\left(z^{(i)}\right)\right)\right]$$

Learning **w**

$$J(\boldsymbol{w}) = \sum_{i=1}^{n}\left[-y^{(i)}\log\left(\phi\left(z^{(i)}\right)\right) - \left(1-y^{(i)}\right)\log\left(1-\phi\left(z^{(i)}\right)\right)\right]$$

- Let us take a look at the cost that we calculate for one single training sample

$$J\left(\phi(z), y; \boldsymbol{w}\right) = -y\log\left(\phi(z)\right) - (1-y)\log\left(1-\phi(z)\right)$$

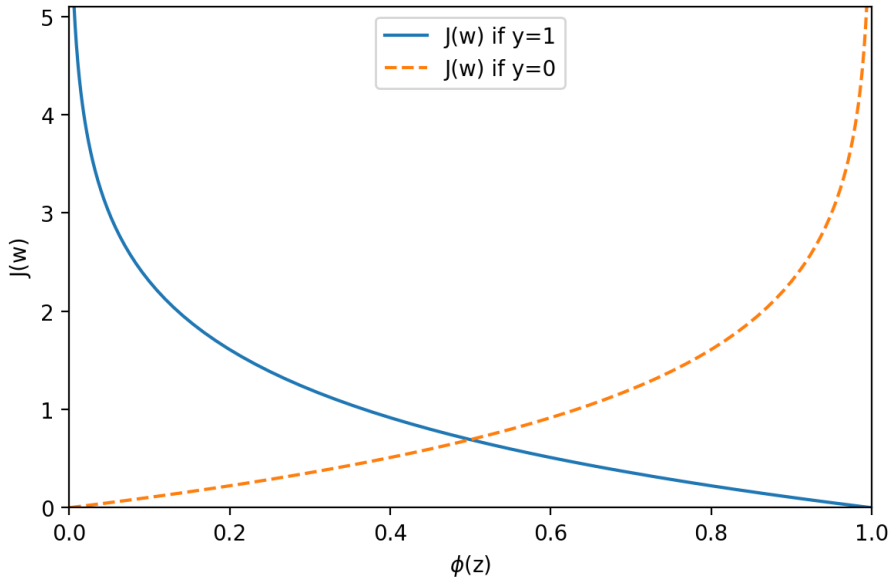
- We can see that the first term is zero if y = 0, and the second term is zero if y = 1

$$J\left(\phi(z), y; \boldsymbol{w}\right) = \begin{cases} -\log\left(\phi(z)\right) & if\ y = 1 \\ -\log\left(1-\phi(z)\right) & if\ y = 0 \end{cases}$$

Plotting our new J

$$J\left(\phi\left(z\right),y;\boldsymbol{w}\right)=\begin{cases}-\log\left(\phi\left(z\right)\right) & \text{if } y=1\\ -\log\left(1-\phi\left(z\right)\right) & \text{if } y=0\end{cases}$$

```
def cost_1(z):
    return - np.log(sigmoid(z))
def cost_0(z):
    return - np.log(1 - sigmoid(z))
z = np.arange(-10, 10, 0.1)
phi_z = sigmoid(z)
c1 = [cost_1(x) for x in z]
plt.plot(phi_z, c1, label='J(w) if y=1')
c0 = [cost_0(x) for x in z]
plt.plot(phi_z, c0, linestyle='--', label='J(w) if y=0')
plt.ylim(0.0, 5.1)
plt.xlim([0, 1])
plt.xlabel('$\phi$(z)')
plt.ylabel('J(w)')
plt.legend(loc='best')
plt.tight_layout()
plt.show()
```



From Adaline to Logistic Regression

- If we were to implement logistic regression ourselves, we could simply substitute the cost function J in our Adaline implementation from the previous chapter